Regularização de sinistros

Fundamentos de Programação — Trabalho individual (Proposta 10)
Mestrado em Business Analytics — Executive · [Nome do estudante] · [N.º de estudante]

1. Contexto do problema e dados

Uma seguradora pretende acompanhar os processos de regularização de sinistros por ramo e por canal de participação, identificando atrasos e valores pendentes para apoiar a organização do trabalho da equipa. A análise **não visa automatizar decisões sobre sinistros**, mas sinalizar onde o acompanhamento operacional se justifica.

O ficheiro `dados_trabalho_10_seguros.csv` contém 12 sinistros fictícios (S001–S012), três por ramo (Automovel, Habitacao, Viagem, ResponsabilidadeCivil) e três por canal (Agente, Corretor, Digital). As variáveis são: ramo, canal, valor reclamado, valor regularizado, dias de resolução e estado (Fechado ou Pendente).

2. Preparação e qualidade dos dados

Os dados foram lidos com `Path` e `pd.read_csv()`. A verificação de qualidade (tipos, valores em falta e duplicados) mostrou o seguinte:

- **Tipos:** os valores monetários e os dias de resolução são inteiros; ramo, canal, estado e identificador são texto.
- **Valores em falta:** nenhum em qualquer coluna.
- **Duplicados:** nenhuma linha duplicada.
- **Categorias:** o estado só toma os valores Fechado e Pendente; os quatro ramos coincidem com os da regra de limites.

Mesmo sem problemas nos dados atuais, o código foi escrito de forma defensiva: os nomes de ramo passam por uma função de normalização (por exemplo, “Automóvel” e “Automovel” são equivalentes) e as colunas numéricas são convertidas com `pd.to_numeric(errors='coerce')`, para que o programa continue a funcionar se chegarem dados menos limpos.

3. Decisões de implementação

3.1 Valor pendente

A coluna `valor_pendente` é igual ao valor reclamado quando o estado é Pendente e zero nos restantes casos. A mesma regra está na classe `Sinistro`, no método `valor_pendente()`.

3.2 Regra central: sinistro fora de prazo

Os limites de regularização são 25 dias para Automovel e Habitacao, 10 dias para Viagem e 35 dias para ResponsabilidadeCivil. A regra foi desenhada em pseudocódigo antes de ser implementada:

```
limite_regularizacao(ramo):
    devolver LIMITES[normalizar(ramo)]     # None se o ramo for desconhecido

cumpre_prazo(ramo, dias_resolucao):
    limite <- limite_regularizacao(ramo)
    SE limite = None OU dias_resolucao em falta:
        devolver VERDADEIRO                # sem informação, não se assinala atraso
    devolver dias_resolucao <= limite
fora_prazo <- NAO cumpre_prazo(ramo, dias_resolucao)    # para cada sinistro
```

Dois pressupostos merecem destaque. Primeiro, o limite é inclusivo: um sinistro resolvido exatamente no último dia ainda cumpre o prazo (por exemplo, Habitacao com 25 dias). Segundo, na ausência de limite ou de dias de resolução não se assinala atraso, para evitar falsos alarmes; com os dados atuais esta situação não ocorre.

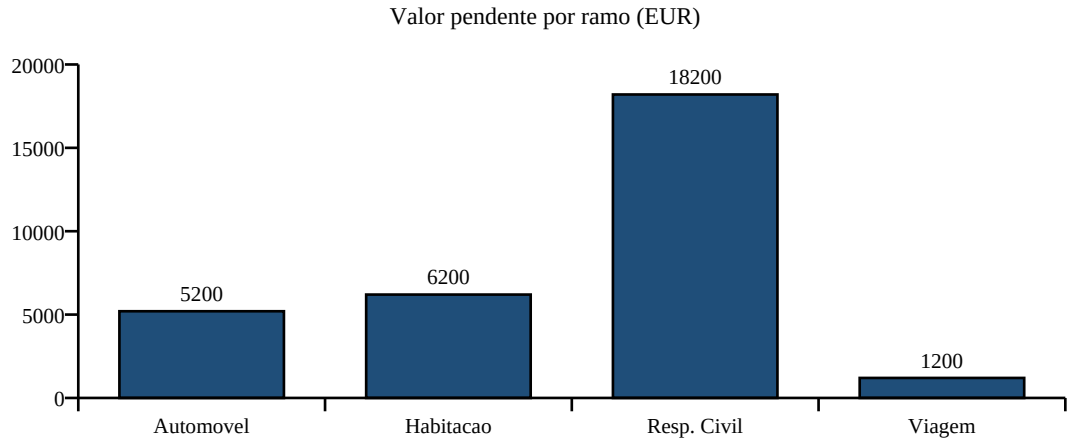
3.3 Restantes componentes

- **Funções próprias:** `normalizar_ramo`, `limite_regularizacao` e `cumpre_prazo`.
- **Resumo por ramo:** `groupby` com agregações nomeadas (contagem, somas, média de dias e percentagem fora de prazo).
- **Classe `Sinistro`:** atributos id, ramo, estado e valor reclamado, com o método `valor_pendente()`. É instanciada com o primeiro sinistro pendente do CSV (S002), para o qual devolve 6 200.
- **Exportação:** `to_csv(..., index=False)`, sem índice técnico.

4. Resultados

4.1 Resumo por ramo

Ramo	N.º sinistros	Reclamado	Regularizado	Pendente	Dias médios	% fora prazo
Automovel	3	11 100	5 400	5 200	29,33	66,67
Habitacao	3	13 800	7 100	6 200	24,00	33,33
ResponsabilidadeCivil	3	25 800	7 000	18 200	46,00	100,00
Viagem	3	2 830	1 500	1 200	10,00	33,33
Total	12	53 530	21 000	30 800	—	58,33 (7 de 12)



4.2 Processos prioritários

São prioritários os sinistros pendentes *ou* fora de prazo. Resultaram 7 processos: os 5 pendentes (todos fora de prazo) e 2 fechados que foram regularizados fora de prazo (S008 e S005). A lista foi ordenada por valor pendente e dias de resolução, ambos de forma decrescente, e exportada para `sinistros_prioritarios.csv`.

Sinistro	Ramo	Canal	Pendente	Dias	Limite	Estado
S012	ResponsabilidadeCivil	Corretor	9 800	55	35	Pendente
S004	ResponsabilidadeCivil	Agente	8 400	45	35	Pendente
S002	Habitacao	Digital	6 200	32	25	Pendente
S009	Automovel	Corretor	5 200	42	25	Pendente
S007	Viagem	Digital	1 200	14	10	Pendente
S008	ResponsabilidadeCivil	Agente	0	38	35	Fechado
S005	Automovel	Digital	0	28	25	Fechado

Distribuição dos prioritários: por **ramo**, ResponsabilidadeCivil tem 3, Automovel 2, Habitacao 1 e Viagem 1; por **canal**, Digital tem 3 (42,9%), Corretor 2 (28,6%) e Agente 2 (28,6%). Na base completa cada canal pesa 33,3%.

5. Interpretação

5.1 Que ramo precisa de maior atenção operacional?

ResponsabilidadeCivil, com base em vários indicadores concordantes:

- **Atrasos:** 100% dos sinistros estão fora de prazo, contra 66,7% em Automovel e 33,3% em Habitacao e Viagem.
- **Duração:** 46 dias em média, 11 acima do limite de 35 dias. Em Automovel a média é 29,3 (limite 25) e em Habitacao 24,0 (limite 25).
- **Valor pendente:** 18 200 €, cerca de 59% do total pendente (30 800 €), embora o ramo represente 48% do valor reclamado.
- **Prioridades:** concentra 3 dos 7 processos prioritários, incluindo os dois maiores valores pendentes (S012 e S004).

Automovel surge em segundo lugar (2 de 3 fora de prazo). Viagem tem a duração média mais baixa, mas o limite é também o mais apertado (10 dias), pelo que 1 de 3 fora de prazo é relevante em proporção, ainda que o valor envolvido seja pequeno.

5.2 Concentração por canal ou ramo

A concentração é mais clara por **ramo** do que por canal. RC reúne 3 dos 7 prioritários e 59% do valor pendente. Por canal, o Digital tem uma ligeira sobre-representação (3 de 7 face a um terço da base), mas com 12 sinistros a diferença corresponde a um único processo e não permite concluir nada. O cruzamento canal × ramo mostra que os 3 prioritários de RC estão no Agente (S004 e S008) e no Corretor (S012), isto é, o problema parece associado ao ramo e não a um canal específico.

5.3 Variáveis adicionais para interpretar atrasos

Para interpretar os atrasos sem atribuir automaticamente responsabilidade a clientes ou equipas, seriam úteis variáveis de contexto, e não de culpabilização:

- **Complexidade do sinistro:** existência de peritagem, litígio, número de intervenientes ou de documentos em falta.
- **Datas por etapa** (participação, abertura, peritagem, decisão, pagamento), para saber onde o processo ficou parado.
- **Motivo de pendência:** à espera de documentação, de terceiros, de perícia ou de decisão interna.
- **Carga de trabalho e capacidade** da equipa no período, para separar atraso individual de sobrecarga.
- **Data de referência:** para os pendentes, dias decorridos até à data da análise.
- **Indicadores agregados** (por ramo, canal e período) e não decisões sobre casos individuais, de acordo com o objetivo do trabalho.

6. Conclusões, limitações e recomendação

Conclusões. Sete dos 12 sinistros (58,3%) estão fora de prazo e 30 800 € continuam pendentes. ResponsabilidadeCivil concentra o maior problema, enquanto o canal tem um efeito pouco evidente.

Limitações.

- Amostra muito pequena (12 sinistros, 3 por ramo): as percentagens variam 33 pontos com um só processo e não são generalizáveis.
- Nos sinistros pendentes, `dias_resolucao` é provavelmente tempo decorrido e não duração final, pelo que o atraso pode estar subestimado.
- Os limites são fixos por ramo e não distinguem a complexidade dos casos.
- Dados fictícios e sem datas, impedindo análise temporal.

Recomendação. Começar o acompanhamento operacional por ResponsabilidadeCivil, em particular pelos sinistros S012 e S004 (18 200 € em conjunto), seguindo-se S009 (Automovel) e S002 (Habitacao), de acordo com a lista prioritária exportada. Em paralelo, recolher as variáveis do ponto 5.3 e repetir a análise com mais dados antes de concluir sobre diferenças entre canais. A lista serve para organizar o trabalho da equipa e não para decidir sobre sinistros individuais.

Anexo — Código (analise_sinistros.py)

```
"""Proposta 10 - Regularização de sinistros."""
from pathlib import Path

import pandas as pd

# Pseudocódigo da regra central (fora_prazo):
#   limite <- limite_regularizacao(ramo)
#   SE limite ou dias_resolucao em falta: considerar dentro do prazo
#   cumpre_prazo <- dias_resolucao <= limite
#   fora_prazo <- NÃO cumpre_prazo

COL_ID = "sinistro"
COL_RAMO = "ramo"
COL_CANAL = "canal"
COL_ESTADO = "estado"
COL_RECLAMADO = "valor_reclamado"
COL_REGULARIZADO = "valor_regularizado"
COL_DIAS = "dias_resolucao"

LIMITES = {
    "Automovel": 25,
    "Habitacao": 25,
    "Viagem": 10,
    "ResponsabilidadeCivil": 35,
}

MAPEAMENTO_RAMO = {
    "Automovel": "Automovel",
    "Automóvel": "Automovel",
    "Habitacao": "Habitacao",
    "Habitação": "Habitacao",
    "Viagem": "Viagem",
    "Responsabilidade Civil": "ResponsabilidadeCivil",
    "ResponsabilidadeCivil": "ResponsabilidadeCivil",
}


def normalizar_ramo(ramo):
    """Padroniza nomes de ramos para comparar com os limites."""
    if pd.isna(ramo):
        return ramo
    valor = str(ramo).strip()
    return MAPEAMENTO_RAMO.get(valor, valor)


def limite_regularizacao(ramo):
    """Devolve o limite em dias do ramo (None se desconhecido)."""
    return LIMITES.get(normalizar_ramo(ramo))


def cumpre_prazo(ramo, dias_resolucao):
    """True se os dias não excedem o limite do ramo.
    Pressuposto: sem limite ou sem dias, não se assinala atraso."""
    limite = limite_regularizacao(ramo)
    if limite is None or pd.isna(dias_resolucao):
        return True
    return dias_resolucao <= limite


class Sinistro:
    """Representa um sinistro e calcula o respetivo valor pendente."""

    def __init__(self, id_sinistro, ramo, estado, valor_reclamado):
        self.id_sinistro = id_sinistro
        self.ramo = ramo
        self.estado = estado
        self.valor_reclamado = valor_reclamado

    def valor_pendente(self):
        return self.valor_reclamado if self.estado == "Pendente" else 0


def main():
    pd.set_option("display.width", 200)
```

```
    pd.set_option("display.max_columns", None)

    # 1. Leitura e qualidade dos dados
    base_dir = Path(__file__).resolve().parent
    csv_file = base_dir / "dados_trabalho_10_seguros.csv"

    if not csv_file.exists():
        raise FileNotFoundError(
            f"Ficheiro de dados não encontrado: {csv_file}\n"
            "Verifique se o CSV está na mesma pasta do script."
        )

    df = pd.read_csv(csv_file)
    df[COL_RAMO] = df[COL_RAMO].map(normalizar_ramo)
    df[COL_DIAS] = pd.to_numeric(df[COL_DIAS], errors="coerce")
    df[COL_RECLAMADO] = pd.to_numeric(df[COL_RECLAMADO], errors="coerce").fillna(0)
    df[COL_REGULARIZADO] = pd.to_numeric(df[COL_REGULARIZADO], errors="coerce").fillna(0)

    print(df.dtypes)
    print(df.isna().sum())
    print("Duplicados:", df.duplicated().sum())
    print(df[COL_ESTADO].unique(), df[COL_RAMO].unique())

    # 2. valor_pendente
    df["valor_pendente"] = df[COL_RECLAMADO].where(
        df[COL_ESTADO].astype(str).str.strip().eq("Pendente"), 0
    )

    # 3. fora_prazo
    df["fora_prazo"] = ~df.apply(
        lambda linha: cumpre_prazo(linha[COL_RAMO], linha[COL_DIAS]), axis=1
    )

    # 4. Resumo por ramo
    resumo = df.groupby(COL_RAMO).agg(
        n_sinistros=(COL_ID, "count"),
        valor_reclamado=(COL_RECLAMADO, "sum"),
        valor_regularizado=(COL_REGULARIZADO, "sum"),
        valor_pendente=("valor_pendente", "sum"),
        dias_medios=(COL_DIAS, "mean"),
        pct_fora_prazo=("fora_prazo", lambda s: s.mean() * 100),
    ).round(2)
    print(resumo)

    # 5. Demonstração da classe
    linha = df[df[COL_ESTADO] == "Pendente"].iloc[0]  # primeiro sinistro pendente
    sinistro = Sinistro(
        linha[COL_ID], linha[COL_RAMO], linha[COL_ESTADO], linha[COL_RECLAMADO]
    )
    print(sinistro.id_sinistro, sinistro.valor_pendente())

    # 6. Exportação dos prioritários
    prioritarios = df[(df[COL_ESTADO].astype(str).str.strip().eq("Pendente")) | df["fora_prazo"]]
    prioritarios = prioritarios.sort_values(
        by=["valor_pendente", COL_DIAS], ascending=[False, False]
    )
    prioritarios.to_csv(base_dir / "sinistros_prioritarios.csv", index=False)

    # Apoio às questões de análise
    print(df[COL_CANAL].value_counts(normalize=True))
    print(prioritarios[COL_CANAL].value_counts(normalize=True))
    print(pd.crosstab(prioritarios[COL_CANAL], prioritarios[COL_RAMO]))


if __name__ == "__main__":
    main()
```